Universidad del Valle

Primer proyecto – ADA II

Programación de tareas con penalización por retraso

Programación dinámica y algoritmos voraces para $1 \,\|\, \sum w_j T_j$

Pablo Nicolás Marín González (2459440) Kevin Steven Rico González (2411024)

Análisis y Diseño de Algoritmos II · Universidad del Valle · 2026-2

Resumen

Se estudia el problema de ordenar n tareas en una sola máquina para minimizar la suma ponderada de sus retrasos, $\sum w_j T_j$. Se demuestra que su versión de decisión pertenece a NP y se discute su NP-dificultad en sentido fuerte. Se propone una solución exacta por programación dinámica sobre subconjuntos, de complejidad $O(2^n n)$, y tres algoritmos voraces (EDD, WSPT y WMDD) de complejidad $O(n \log n)$ u $O(n^2)$. Ambos enfoques se implementan en C++ y se comparan sobre 315 instancias aleatorias con n entre 4 y 24. La programación dinámica coincide con la fuerza bruta en todos los casos verificables y su tiempo se duplica con cada tarea adicional; el voraz WMDD obtiene brechas medianas de 0.2 % a 1.2 % cuando las fechas límite son exigentes, pero ningún voraz garantiza el óptimo.

1. Análisis teórico del problema

1.1. Descripción informal

Una sola máquina (o persona) debe ejecutar n tareas, una a la vez y sin interrumpirlas. Cada tarea tarda un tiempo, tiene una fecha en la que debería estar terminada y una penalización por cada unidad de tiempo que se entregue tarde. La pregunta es en qué orden ejecutarlas para que la penalización total sea mínima.

1.2. Modelo formal

Definición 1 (Instancia)**.** *Una instancia es un conjunto de tareas* $J = \{1, \ldots, n\}$ *donde cada tarea* j *tiene una* duración $p_j \in \mathbb{N}_{>0}$*, una* fecha límite $d_j \in \mathbb{N}$ *y un* peso *o penalización por unidad de retraso* $w_j \in \mathbb{N}$*. Todas las tareas están disponibles en el instante* 0*.*

Definición 2 (Programación y costo)**.** *Una programación es una permutación* $\pi = (\pi_1, \ldots, \pi_n)$ *de* J*. La tarea en la posición* k *termina en*

$$C_{\pi_k} = \sum_{i=1}^{k} p_{\pi_i},$$

su retraso *es* $T_j = \max(0,\ C_j - d_j)$ *y el costo de* π *es*

$$\text{costo}(\pi) = \sum_{j=1}^{n} w_j\, T_j = \sum_{j=1}^{n} w_j \max(0,\ C_j - d_j).$$

Problema de optimización. Encontrar $\pi^* = \arg\min_{\pi \in S_n} \text{costo}(\pi)$, donde S_n es el conjunto de las $n!$ permutaciones. En la notación de Graham se escribe $1 \,\|\, \sum w_j T_j$: una máquina (1), sin restricciones adicionales (campo vacío) y objetivo de retraso ponderado total.

No es necesario considerar tiempos ociosos ni interrupciones: dejar la máquina quieta solo retrasa las tareas siguientes, y como todas están disponibles desde $t = 0$, toda programación óptima puede tomarse sin huecos. Así, el espacio de soluciones es exactamente S_n.

Interpretación elegida. El enunciado habla de una "penalización asociada" al retraso. Se adoptó la lectura en la que la penalización es *proporcional al retraso* ($w_j T_j$). La alternativa, una multa fija si la tarea llega tarde ($\sum w_j U_j$), es un problema distinto, más cercano a la mochila. Sin fechas límite el problema sería $\sum w_j C_j$, que la regla de Smith resuelve en tiempo polinomial (Sección 3.2); por eso las fechas límite son esenciales para la dificultad.

1.3. Ejemplo

Considérense tres tareas:

Tarea	p_j	d_j	w_j
A	3	4	2
B	2	2	5
C	4	6	1

Para $\pi = (A, B, C)$: $C_A = 3$, $C_B = 5$, $C_C = 9$; los retrasos son 0, 3 y 3, y el costo es $2{\cdot}0+5{\cdot}3+1{\cdot}3 = 18$. La Tabla 1 enumera las $3! = 6$ permutaciones: la óptima es (B, A, C) con costo 5. Poner primero la tarea más costosa de retrasar (B) reduce el costo a menos de un tercio.

Tabla 1: Todas las programaciones del ejemplo.

π	(C_j) en orden	(T_j) en orden	costo
A, B, C	$3, 5, 9$	$0, 3, 3$	18
A, C, B	$3, 7, 9$	$0, 1, 7$	36
$\mathbf{B, A, C}$	$2, 5, 9$	$0, 1, 3$	**5**
B, C, A	$2, 6, 9$	$0, 0, 5$	10
C, A, B	$4, 7, 9$	$0, 3, 7$	41
C, B, A	$4, 6, 9$	$0, 4, 5$	30

1.4. Pertenencia a NP

NP es una clase de *problemas de decisión*, por lo que se considera la versión:

> RetrasoPonderado: dada una instancia y un entero K, ¿existe una permutación π con costo$(\pi) \le K$?

Proposición 1. *RetrasoPonderado $\in$ NP.*

Demostración. El certificado es la permutación π, de tamaño $O(n \log n)$ bits, polinomial en la entrada. El verificador comprueba que π sea una permutación de J (marcando cada índice en un arreglo, $O(n)$), recorre π acumulando $C_{\pi_k} = C_{\pi_{k-1}} + p_{\pi_k}$, suma $w_j \max(0, C_j - d_j)$ y compara el total con K. Son $O(n)$ operaciones aritméticas sobre números de tamaño polinomial en la entrada, así que la verificación es polinomial. Si la respuesta es *sí*, el π correcto hace aceptar al verificador; si es *no*, ningún π lo hace. □

El problema de *optimización* no está en NP en sentido estricto, porque no es de decisión, pero es tan difícil como la versión de decisión. Con un algoritmo que resuelva la decisión se encuentra el óptimo por búsqueda binaria sobre $K \in [0, \sum_j w_j \sum_i p_i]$, y conocido el costo óptimo se reconstruye la permutación fijando las posiciones una a una.

1.5. Complejidad computacional

Teorema 1 (Lawler 1977; Lenstra, Rinnooy Kan y Brucker 1977)**.** *$1 \,\|\, \sum w_j T_j$ es NP-difícil en sentido fuerte. En consecuencia, RetrasoPonderado es NP-completo en sentido fuerte.*

La demostración original reduce 3-Partición, que es NP-completo en sentido fuerte, a este problema. Que la dificultad sea *fuerte* tiene una consecuencia concreta para el diseño: a diferencia de la mochila o de $1 \,\|\, \sum w_j U_j$, **no existe un algoritmo pseudo-polinomial**, es decir, polinomial en n y en la magnitud de los números, a menos que P = NP. Una tabla indexada por el tiempo, $O(n \sum p_j)$, no puede funcionar en general. Por eso la programación dinámica de la Sección 2 es exponencial en n: es lo esperable para un problema fuertemente NP-difícil.

Tamaño del espacio de búsqueda. Hay $n!$ programaciones. Por la fórmula de Stirling, $n! \approx \sqrt{2\pi n}\,(n/e)^n$, que crece más rápido que cualquier c^n. Para $n = 20$ hay unos 2.4×10^{18} órdenes; evaluar cada uno en $O(n)$ con 10^9 operaciones por segundo tomaría más de 1500 años.

2. Solución por programación dinámica

2.1. Razonamiento que lleva a la formulación

El punto de partida es la fuerza bruta, que considera los $n!$ órdenes. La pregunta clave es: *¿qué información de un prefijo de la programación se necesita realmente para continuar?*

Supóngase que ya se decidió qué tareas van primero, el conjunto S, aunque no su orden interno. Se observan dos hechos:

1. Cualquiera sea su orden interno, las tareas de S ocupan exactamente el intervalo $[0, P(S)]$, con $P(S) = \sum_{j \in S} p_j$. Las tareas fuera de S empiezan todas después de $P(S)$, así que su costo **no depende** del orden interno de S.
2. La última tarea ejecutada dentro de S termina en $P(S)$, también independientemente del orden de las demás.

Por lo tanto, para el futuro solo importa *qué* tareas ya se ejecutaron, no *en qué orden*. Esto reduce la información de estado de $n!$ órdenes a 2^n subconjuntos. Es la misma idea del algoritmo de Held–Karp para el problema del viajante.

Lema 1. *En cualquier orden de las tareas de S ejecutado desde $t = 0$ sin huecos, la última tarea termina en $P(S)$.*

Demostración. Su tiempo de finalización es la suma de las duraciones de todas las tareas ejecutadas hasta ella inclusive, que son exactamente las de S. □

2.2. Modelo matemático

Definición 3 (Estado)**.** *Para $S \subseteq J$, $f(S)$ es el costo mínimo de ejecutar* exactamente *las tareas de S al inicio de la programación, en el mejor orden posible.*

Recurrencia. Se elige cuál tarea $j \in S$ va de última. Por el Lema 1, termina en $P(S)$, y lo anterior es un subproblema sobre $S \setminus \{j\}$:

$$\boxed{\begin{aligned} &f(\varnothing) = 0, \\ &f(S) = \min_{j \in S} \Big[f(S \setminus \{j\}) + w_j \max\big(0,\, P(S) - d_j\big) \Big]. \end{aligned}} \tag{1}$$

La respuesta es $f(J)$. Para reconstruir el orden se guarda, para cada S, la tarea $u(S)$ que alcanza el mínimo, y se retrocede desde J: $u(J)$ es la última tarea, $u(J \setminus \{u(J)\})$ la penúltima, etc.

Teorema 2 (Corrección)**.** *Para todo $S \subseteq J$, $f(S)$ calculado con* (1) *es igual al costo mínimo de ordenar las tareas de S desde $t = 0$.*

Demostración. Por inducción sobre $|S|$. Si $S = \varnothing$ el costo es 0. Sea $|S| \geq 1$ y llámese $\mathrm{OPT}(S)$ al costo mínimo verdadero.

($\geq$) Sea σ un orden óptimo de S y j su última tarea. Por el Lema 1, j cuesta $w_j \max(0, P(S) - d_j)$. Las demás forman un orden de $S \setminus \{j\}$ que empieza en $t = 0$, de costo $\geq \mathrm{OPT}(S \setminus \{j\}) = f(S \setminus \{j\})$ por hipótesis inductiva. Entonces $\mathrm{OPT}(S) \geq f(S \setminus \{j\}) + w_j \max(0, P(S) - d_j) \geq f(S)$.

($\leq$) Sea j la tarea que minimiza (1). Tomando un orden óptimo de $S \setminus \{j\}$, cuyo costo es $f(S \setminus \{j\})$ por hipótesis inductiva, y poniendo j al final, se obtiene un orden de S con costo exactamente $f(S)$. Así $\mathrm{OPT}(S) \leq f(S)$. □

La demostración evidencia las dos propiedades que justifican la programación dinámica: **subestructura óptima**, porque el prefijo de un orden óptimo es óptimo para su conjunto, y **subproblemas superpuestos**, porque el mismo $f(S)$ se usa desde todos los $S \cup \{j\}$, hasta $n - |S|$ veces.

2.3. Ejemplo: tabla completa

La Tabla 2 aplica (1) al ejemplo de la Sección 1.3. Cada candidato es $f(S \setminus \{j\}) + w_j \max(0, P(S) - d_j)$ con j como última tarea.

Tabla 2: Programación dinámica sobre el ejemplo A, B, C.

S	$P(S)$	candidatos (j: valor)	$f(S)$
$\varnothing$	0	–	0
$\{A\}$	3	A: $0 + 2 \cdot 0 = 0$	0
$\{B\}$	2	B: $0 + 5 \cdot 0 = 0$	0
$\{C\}$	4	C: $0 + 1 \cdot 0 = 0$	0
$\{A, B\}$	5	A: $0 + 2 \cdot 1 = \mathbf{2}$; B: $0 + 5 \cdot 3 = 15$	2
$\{A, C\}$	7	A: $0 + 2 \cdot 3 = 6$; C: $0 + 1 \cdot 1 = \mathbf{1}$	1
$\{B, C\}$	6	B: $0 + 5 \cdot 4 = 20$; C: $0 + 1 \cdot 0 = \mathbf{0}$	0
$\{A, B, C\}$	9	A: $0 + 10 = 10$; B: $1 + 35 = 36$; C: $2 + 3 = \mathbf{5}$	5

Reconstrucción:

$$u(\{A, B, C\}) = C, \quad u(\{A, B\}) = A, \quad u(\{B\}) = B.$$

El orden es (B, A, C) con costo 5, igual al de la Tabla 1. Se evaluaron 12 candidatos en lugar de las $6 \cdot 3 = 18$ operaciones de la fuerza bruta; la diferencia crece de forma exponencial con n.

2.4. Algoritmo

Los subconjuntos se representan como máscaras de bits: el bit j de un entero S vale 1 si la tarea j está en S. Si $S' \subset S$, entonces $S' < S$ como entero, así que basta recorrer las máscaras en orden creciente para que todo $f(S \setminus \{j\})$ esté calculado antes de usarlo.

Algoritmo 1 Programación dinámica sobre subconjuntos

Entrada: tareas (p_j, d_j, w_j), $j = 0, \ldots, n-1$
Salida: orden óptimo y su costo

```
f[0] ← 0
para S ← 1 hasta 2^n − 1 hacer
    P ← Σ_{j∈S} p_j ;   f[S] ← ∞
    para cada j ∈ S hacer
        c ← f[S \ {j}] + w_j · max(0, P − d_j)
        si c < f[S] entonces f[S] ← c ;  u[S] ← j
S ← 2^n − 1 ;  π ← ()
mientras S ≠ 0 hacer
    anteponer u[S] a π ;  S ← S \ {u[S]}
retornar (π, f[2^n − 1])
```

2.5. Complejidad

Tiempo. Hay 2^n estados; para cada uno se calcula $P(S)$ en $O(n)$ y se prueban hasta n candidatos en $O(1)$ cada uno. Total

$$T(n) = \Theta(2^n \cdot n).$$

Más precisamente, el número de candidatos es $\sum_S |S| = \sum_{k=0}^{n} k \binom{n}{k} = n\, 2^{n-1}$. La reconstrucción cuesta $O(n)$.

Espacio. Se guardan $f[S]$ (entero de 64 bits) y $u[S]$ (entero de 8 bits) para cada S: $\Theta(2^n)$, unos $9 \cdot 2^n$ bytes. Para $n = 24$ son cerca de 150 MB, que se tomó como límite práctico de la implementación.

Comparación con la fuerza bruta. La fuerza bruta cuesta $\Theta(n \cdot n!)$. El cociente

$$\frac{n \cdot n!}{n \cdot 2^n} = \frac{n!}{2^n} = \prod_{k=1}^{n} \frac{k}{2}$$

crece sin cota: para $n = 10$ es $\approx 3.5 \times 10^3$ y para $n = 20$ es $\approx 2.3 \times 10^{12}$. La programación dinámica sigue siendo

exponencial, como se espera de un problema fuertemente NP-difícil, pero mueve la frontera de lo resoluble de $n \approx 11$ a $n \approx 25$.

3. Solución mediante enfoques voraces

Un algoritmo voraz construye la programación eligiendo en cada paso la tarea que *parece* mejor según un criterio local, sin reconsiderar decisiones. Se implementaron tres criterios. Los dos primeros son reglas clásicas que ignoran uno de los parámetros; el tercero, WMDD, combina ambos y es el voraz principal de este trabajo. En todos los casos los empates se resuelven a favor de la tarea de menor índice.

3.1. EDD: fecha límite más cercana

Criterio: ordenar por d_j creciente. **Complejidad:** $O(n \log n)$.

Justificación: atender primero lo que vence primero. Por el teorema de Jackson (1955), EDD minimiza el retraso *máximo*; en particular, si existe una programación en la que ninguna tarea se retrasa, EDD la encuentra.

Limitación: ignora por completo los pesos w_j y las duraciones, así que una tarea barata con fecha cercana puede desplazar a una muy costosa.

3.2. WSPT: regla de Smith

Criterio: ordenar por w_j/p_j decreciente, es decir, mayor penalización por unidad de tiempo ocupado. **Complejidad:** $O(n \log n)$.

Proposición 2 (Intercambio adyacente)**.** *Sean i, j tareas consecutivas que empiezan en el instante t, ambas tardías en cualquiera de sus dos órdenes, lo que ocurre si $d_i \le t + p_i$ y $d_j \le t + p_j$. Entonces conviene poner i antes que j si y solo si $w_i/p_i \ge w_j/p_j$.*

Demostración. Si ambas son tardías, $T = C - d$. Con i primero, el costo del par es $w_i(t+p_i-d_i)+w_j(t+p_i+p_j-d_j)$; con j primero, $w_j(t+p_j-d_j)+w_i(t+p_j+p_i-d_i)$. La resta (primero menos segundo) es $w_j p_i - w_i p_j$, que es ≤ 0 si y solo si $w_i/p_i \ge w_j/p_j$. El resto de la programación no cambia porque el par termina en $t+p_i+p_j$ en ambos casos. □

Observación 1. *Si todas las fechas límite son 0, todas las tareas son tardías y el objetivo se vuelve $\sum w_j C_j - cte$. Aplicando la Proposición 2 a cualquier par adyacente fuera del orden WSPT se demuestra que WSPT es* óptimo *en ese caso (Smith, 1956). Por eso el problema sin fechas límite está en P.*

Limitación: ignora las fechas límite; puede adelantar una tarea con mucha holgura y atrasar otra que estaba por vencer.

3.3. WMDD: fecha límite modificada ponderada

Criterio: en cada paso, con t igual al tiempo ya ocupado, elegir la tarea pendiente que minimice

$$\rho_j(t) = \frac{\max\left(p_j,\ d_j - t\right)}{w_j}.$$

Las tareas con $w_j = 0$ no generan costo, tienen prioridad $+\infty$ y van al final.

Justificación del diseño. ρ_j cambia de régimen según la holgura $d_j - t$ de cada tarea:

- Si la tarea ya no puede terminar a tiempo ($d_j - t \le p_j$), entonces $\rho_j = p_j/w_j$: es WSPT, que la Proposición 2 justifica entre tareas tardías.
- Si aún tiene holgura ($d_j - t > p_j$), entonces $\rho_j = (d_j - t)/w_j$: es EDD ponderado, donde se adelanta lo que vence antes, ajustado por lo que cuesta retrasarlo.

El criterio es *dinámico*: como depende de t, debe recalcularse en cada paso. Por eso cuesta $O(n^2)$, con n pasos y n candidatos por paso. No se usan números de punto flotante: $a/w < a'/w'$ se compara como $a\,w' < a'\,w$, lo que con los límites de entrada ($a \le 10^9$, $w \le 10^6$) cabe en 64 bits.

Algoritmo 2 Voraz WMDD

Entrada: tareas (p_j, d_j, w_j), $j = 1, \ldots, n$
Salida: un orden π (no necesariamente óptimo)

```
1: R ← {1,...,n};  t ← 0;  π ← ()
2: mientras R ≠ ∅ hacer
3:     k ← arg mín_{j∈R} máx(p_j, d_j − t)/w_j
       (empates: menor índice; w_j = 0 cuenta como +∞)
4:     agregar k al final de π
5:     t ← t + p_k;  R ← R \ {k}
6: retornar π
```

La línea 3 es la única decisión del algoritmo y nunca se revisa después. Esa es la diferencia esencial con el Algoritmo 1, que para cada conjunto compara *todas* las tareas que podrían ir de últimas.

3.4. ¿Garantizan optimalidad?

No. Ningún voraz polinomial puede garantizarla: si alguno lo hiciera se tendría un algoritmo polinomial para un problema NP-difícil, y por tanto P = NP. Basta un contraejemplo concreto.

Contraejemplo. Sean las tareas

Tarea	1	2	3	4
p_j	4	5	6	4
d_j	9	7	6	5
w_j	5	3	3	2

El óptimo, verificado por fuerza bruta y por programación dinámica, es $(2, 1, 3, 4)$ con costo 55: las tareas 2 y 1 terminan en 5 y 9, a tiempo, y 3 y 4 se retrasan 9 y 14 unidades ($3 \cdot 9 + 2 \cdot 14 = 55$).

WMDD en $t = 0$ calcula $\rho = (9/5,\ 7/3,\ 6/3,\ 5/2) = (1.8,\ 2.33,\ 2,\ 2.5)$ y elige la tarea 1, por su peso alto. Pero la tarea 1 tenía holgura de sobra: podía ir segunda y aun así terminar en $9 = d_1$. Elegirla primero retrasa la 2 en 2 unidades. El resultado es $(1, 2, 3, 4)$ con costo 61, una brecha de 10.9 %. WSPT produce el mismo orden (61) y EDD obtiene $(4, 3, 2, 1)$ con costo 86 (+56.4 %).

El error tiene una causa clara: el voraz mira solo el instante actual y no *prevé* que retrasar una tarea con holgura no cuesta nada. La programación dinámica sí lo considera, porque evalúa todos los conjuntos posibles de tareas previas.

3.5. Comparación conceptual con la programación dinámica

Tabla 3: Programación dinámica frente a voraz.

	Programación dinámica	Voraz (WMDD)
Decisiones	considera todas las tareas que pueden ir de últimas en cada conjunto	una elección local e irrevocable por paso
Optimalidad	garantizada (teorema de corrección)	no garantizada (contraejemplo)
Tiempo	$\Theta(2^n n)$	$\Theta(n^2)$
Espacio	$\Theta(2^n)$	$\Theta(n)$
Tamaño práctico	$n \leq 24$	miles de tareas
Principio	subestructura óptima y subproblemas superpuestos	criterio de intercambio local (Prop. 2)

Ambos enfoques parten de observaciones estructurales distintas. La programación dinámica aprovecha que el costo futuro depende solo del *conjunto* ya programado, y por eso puede explorar todas las opciones sin repetir trabajo. El voraz aprovecha que, entre dos tareas tardías adyacentes, el orden correcto lo decide una razón local (w/p). Esa regla es exacta para pares de tareas tardías, pero no captura la interacción global entre holguras, que es justamente lo que hace difícil al problema.

4. Implementación

Ambas soluciones se implementaron en C++17, sin bibliotecas externas, y se compilan con `g++ -O2` mediante un `Makefile`. El código está organizado en módulos pequeños con una interfaz común: cada algoritmo recibe el vector de tareas y devuelve un `Resultado` con el orden (identificadores $1..n$) y su costo.

Archivo	Responsabilidad
`entrada.cpp`	lectura y validación de instancias
`costo.cpp`	costo y detalle de un orden
`dp.cpp`	programación dinámica
`fuerza_bruta.cpp`	verificación por enumeración
`voraz.cpp`	EDD, WSPT y WMDD
`generacion.cpp`	instancias aleatorias
`main.cpp`	programa `tareas`
`experimentos.cpp`	mediciones en CSV

Decisiones de implementación relevantes.

- **Máscaras de 32 bits** para los subconjuntos y recorrido en orden creciente, lo que evita la recursión y la memoización con mapas. $P(S)$ se calcula en el mismo ciclo que los candidatos, sin un arreglo adicional.
- **Arreglo de predecesores de 8 bits** ($u[S] \in [0, 23]$), que reduce la memoria de 16 a 9 bytes por estado.
- **Aritmética entera exacta**: costos en `long long` y comparaciones de cocientes por producto cruzado. Los límites de entrada ($n \leq 1000$, $p, w \leq 10^6$, $d \leq 10^9$) garantizan que no haya desbordamiento; el peor caso, $\approx 5 \times 10^{17}$, se verifica en las pruebas.
- **Validación de la entrada** con mensajes que indican la línea del error. Se toleran comentarios, líneas vacías, fines de línea de Windows y la marca BOM, para poder usar archivos preparados en otro sistema durante la sustentación.
- **Límites explícitos**: la fuerza bruta rechaza $n > 10$ y la programación dinámica $n > 24$ con un mensaje claro, en vez de agotar la memoria.

Uso. `./tareas archivo.txt [--algoritmo dp|fb|edd|wspt|wmdd|todos] [--detalle]` muestra para cada algoritmo el orden, el costo y el tiempo y, con `--detalle`, la tabla $C_j, T_j, w_j T_j$ de cada tarea. Al final reporta la brecha de cada voraz frente al óptimo. El archivo de entrada contiene n y luego una línea $p_j \ \ d_j \ \ w_j$ por tarea. `./generador n semilla` crea instancias reproducibles y `./experimentos` produce los datos de la Sección 5.

5. Pruebas y resultados

5.1. Pruebas de corrección

El conjunto automático (`make test`, 2558 comprobaciones) incluye:

- Costos calculados a mano (ejemplo A, B, C y su detalle).
- **Programación dinámica igual a fuerza bruta** en 300 instancias aleatorias con $n \leq 8$: el costo coincide en todas.
- Todo orden devuelto es una permutación de $1..n$ y su costo recalculado coincide con el reportado.
- Ningún voraz obtiene un costo menor que el de la programación dinámica, y existe al menos una instancia donde WMDD no es óptimo.
- Casos borde: $n = 0$, pesos nulos, valores máximos, límites de n y archivos mal formados, que deben terminar con código de error 1.

5.2. Casos diseñados por el equipo

Además de las pruebas automáticas, se diseñaron cuatro instancias (carpeta `ejemplos/`) para ilustrar comportamientos concretos. La Tabla 4 muestra el costo obtenido por cada algoritmo al ejecutar `./tareas` sobre cada archivo.

Tabla 4: Costo de cada algoritmo en los casos del equipo (FB = fuerza bruta, PD = programación dinámica; en negrita, los que alcanzan el óptimo).

Caso	n	FB	PD	EDD	WSPT	WMDD
`ejemplo_basico`	3	**5**	**5**	**5**	**5**	**5**
`todas_a_tiempo`	4	**0**	**0**	**0**	**0**	**0**
`voraz_no_optimo`	4	**55**	**55**	86	61	61
`mediano_15`	15	–	**798**	1495	1028	**798**

- `ejemplo_basico`: el ejemplo de la Sección 1.3. Todos los algoritmos encuentran el orden (B, A, C): con pocas tareas y criterios alineados (B vence primero y es la más costosa de retrasar), los voraces aciertan.
- `todas_a_tiempo`: fechas límite holgadas, así que cualquier orden cuesta 0. Los algoritmos devuelven órdenes distintos, todos óptimos: el óptimo no tiene por qué ser único.
- `voraz_no_optimo`: el contraejemplo de la Sección 3.4. La fuerza bruta devuelve $(2, 1, 3, 4)$ y la programación dinámica $(2, 1, 4, 3)$, ambos de costo 55: dos óptimos distintos que cada algoritmo desempata de forma diferente.
- `mediano_15`: instancia aleatoria (`./generador 15 2024`) fuera del alcance de la fuerza bruta ($15! \approx 1.3 \times 10^{12}$ órdenes). La programación dinámica la resuelve en milisegundos y WMDD también llega al óptimo, mientras que EDD casi duplica el costo (+87 %).

También se comprobó el manejo de entradas inválidas: un archivo con menos tareas de las declaradas, valores negativos o no numéricos, y la petición de programación dinámica con $n = 25$ terminan con un mensaje que indica la causa y código de salida 1.

5.3. Diseño experimental

Se generaron instancias con $p_j, w_j \sim U\{1, \ldots, 10\}$ y $d_j \sim U\{0, \ldots, \lfloor \alpha \sum p_i \rfloor\}$, donde α controla la *holgura* de las fechas límite: alta ($\alpha = 1$), media (0.6) y baja (0.3, la mayoría de las tareas termina tarde). Para cada $n \in \{4, \ldots, 24\}$ se usaron 5 semillas y las 3 holguras, es decir $21 \cdot 5 \cdot 3 = 315$ instancias. La fuerza bruta solo se ejecutó con $n \leq 10$. Los tiempos se midieron con `std::chrono::steady_clock`, en un solo hilo, en un computador con Linux.

5.4. Tiempo de ejecución

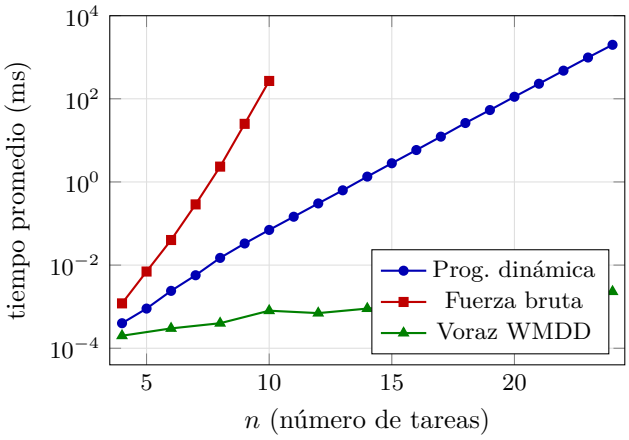


Figura 1: Tiempo promedio por instancia (escala logarítmica).

La Figura 1 y la Tabla 5 confirman el análisis asintótico:

- En escala logarítmica, la programación dinámica es casi una recta: el tiempo **se multiplica por** ≈ 2.05 con cada tarea adicional. El modelo $\Theta(2^n n)$ predice $2(n+1)/n \approx 2.09$ para $n \approx 22$.
- La fuerza bruta se multiplica por $\approx n + 1$, como predice $\Theta(n \cdot n!)$, y su curva se curva hacia arriba. Con $n = 10$ ya es unas 3800 veces más lenta que la programación dinámica; extrapolando, con $n = 14$ tardaría horas.

Tabla 5: Tiempo promedio (ms) y razón de crecimiento al aumentar n en 1.

n	PD	razón	FB	razón
7	0.0057	2.4	0.289	7.2
8	0.0149	2.6	2.34	8.1
9	0.0331	2.2	25.0	10.7
10	0.0701	2.1	268.4	10.8
20	111.8	2.09	–	–
21	230.0	2.06	–	–
22	474.8	2.06	–	–
23	983.2	2.07	–	–
24	1987.3	2.02	–	–

- Los voraces se mantienen por debajo de 0.003 ms en todo el rango; a esta escala el tiempo está dominado por costos fijos (reserva de memoria, medición), no por el n^2.
- Con $n = 24$ la programación dinámica tarda 2 s y usa cerca de 150 MB. Cada tarea extra duplica ambos, así que $n \approx 28$ ya requeriría más de 2 GB.

5.5. Calidad de los voraces

Se define la brecha de un voraz como

$$\text{brecha} = 100 \cdot \frac{\text{costo}_{\text{voraz}} - \text{costo}_{\text{óptimo}}}{\text{costo}_{\text{óptimo}}}.$$

Tabla 6: Brecha (%) de cada voraz frente al óptimo, 105 instancias por holgura.

Voraz	Holgura	media	mediana	máx.	óptimos
EDD	alta	111.1	84.5	450.4	10
	media	102.0	96.3	255.1	1
	baja	72.2	73.0	170.9	1
WSPT	alta	495.7	227.1	10500	3
	media	30.2	29.1	84.2	7
	baja	4.2	4.0	10.8	20
WMDD	alta	115.9	17.6	7500	12
	media	**2.0**	**1.2**	13.1	27
	baja	**0.6**	**0.2**	4.8	44

De la Tabla 6 se concluye:

- **WMDD domina** a los otros dos voraces en todas las holguras. Con holgura media y baja su brecha mediana es de 1.2 % y 0.2 %, y alcanza el óptimo exacto en 71 de 210 instancias.
- **WSPT mejora a medida que la holgura baja**: de 227 % a 4 % de mediana. Es lo que predice la teoría, porque cuando casi todas las tareas son tardías la Proposición 2 se aplica a casi todos los pares.
- **EDD es el peor** cuando hay retrasos, porque ignora los pesos. Su brecha mediana nunca baja de 73 %.
- **Con holgura alta los porcentajes engañan**: el costo óptimo suele ser muy pequeño (pocas tareas tardías), y una diferencia absoluta de pocas unidades produce brechas de miles por ciento (el máximo de 7500 % de WMDD es una instancia con óptimo 1 donde el voraz obtiene 76). La mediana es una medida más representativa que la media en este régimen.

5.6. Comportamiento en función del tamaño de la entrada

- La **programación dinámica** es exacta para cualquier n, pero su costo crece exponencialmente. El límite es la memoria más que el tiempo.
- La **brecha del voraz** no muestra una tendencia monótona con n: la media de WMDD oscila entre 4 % y 30 % para $n \geq 9$, salvo un pico en $n = 15$. Depende mucho más de la estructura de la instancia (la holgura) que de su tamaño. Los picos aislados, como $n = 8$ o $n = 15$, vienen de instancias de holgura alta con óptimo cercano a cero.
- Sí aumenta con n la *probabilidad* de no dar el óptimo: con $n = 4$ WMDD es óptimo en 13 de 15 instancias, mientras que con $n \geq 20$ solo lo fue en 5 de 75, porque hay más pares de tareas donde una decisión local puede equivocarse.

6. Análisis del proceso

Cómo se llegó a cada modelo. El primer paso fue precisar el problema. El enunciado admitía dos lecturas (penalización proporcional al retraso o multa fija), y se eligió la proporcional por ser la más literal. Se comprobó además que sin fechas límite el problema sería polinomial, lo que justificó incluirlas en el modelo.

Para la programación dinámica se partió de la fuerza bruta y se buscó qué parte del prefijo de un orden afecta al resto. Al notar que la última tarea de un conjunto siempre termina en $P(S)$ (Lema 1), el estado natural resultó ser el conjunto y no el orden. Se descartó un estado indexado por el tiempo, $f(k, t)$, porque no permite saber qué tareas faltan; esa es la razón de fondo por la que no hay solución pseudo-polinomial.

Para el voraz se analizó primero por qué fallan las reglas simples: EDD ignora w y WSPT ignora d. El argumento de intercambio (Proposición 2) mostró que WSPT es correcto entre tareas tardías, y eso llevó a un criterio que se comporta como WSPT para las tareas tardías y como EDD ponderado para las que aún tienen holgura: WMDD.

Dificultades encontradas.

- **Ambigüedad del enunciado**, resuelta con el análisis de las variantes y de su complejidad.
- **Memoria de la programación dinámica**: guardar costo, predecesor y $P(S)$ en arreglos de 64 bits costaría 24 bytes por estado. Por eso se usó un predecesor de 8 bits y se calculó $P(S)$ dentro del ciclo: 9 bytes por estado, lo que permite llegar a $n = 24$.
- **Archivos de entrada de otros sistemas**: un archivo guardado en Windows como UTF-8 con BOM era rechazado con un error confuso. Se detectó en la revisión del código y se corrigió con una prueba que lo cubre.
- **Tareas con** $w_j = 0$: su prioridad en WMDD implicaría dividir por cero. Se definió prioridad infinita (no cuesta retrasarlas) y se evitó la división con productos cruzados.
- **Verificar la optimalidad**: no basta con probar algunos ejemplos a mano. La fuerza bruta se usó como oráculo sobre cientos de instancias aleatorias.
- **Interpretar la brecha**: la media en porcentaje resultó engañosa con óptimos pequeños, y se complementó con la mediana.

Decisiones de diseño. C++ por su control de memoria y su eficiencia en los ciclos de la programación dinámica; máscaras de bits en lugar de conjuntos explícitos; los mismos módulos para el programa, las pruebas y los experimentos; instancias generadas con semilla fija para que todos los resultados sean reproducibles.

7. Conclusiones

1. **La programación dinámica no elimina la dificultad, la reorganiza.** Pasar de $n!$ órdenes a 2^n conjuntos es una mejora enorme, del orden de 10^{12} para $n = 20$, pero sigue siendo exponencial. Esto es coherente con que el problema sea fuertemente NP-difícil: no cabe esperar una tabla polinomial ni pseudo-polinomial. La ganancia proviene de identificar la información *mínima* suficiente sobre el pasado, el conjunto y no el orden.
2. **La frontera práctica la fija la memoria.** El tiempo se duplica y la memoria también con cada tarea; con $n = 24$ la memoria se acerca al límite razonable antes que el tiempo. En la práctica, el análisis de espacio es tan importante como el de tiempo.
3. **Un voraz es tan bueno como la estructura que explota.** WSPT es exacto cuando todas las tareas son tardías y empeora cuando no lo son; WMDD funciona bien porque su criterio cambia de régimen siguiendo esa misma estructura. La calidad de un voraz no es una propiedad fija: depende de la distribución de las instancias.
4. **La falta de optimalidad es estructural, no un defecto de implementación.** El contraejemplo muestra el mecanismo: una decisión local no puede prever que retrasar una tarea con holgura es gratis. Un voraz que siempre acertara probaría $\mathrm{P} = \mathrm{NP}$.
5. **Los dos enfoques se complementan.** La programación dinámica sirve como referencia exacta para medir los voraces, y estos resuelven en microsegundos instancias que la programación dinámica no puede abordar. Una extensión natural sería combinarlos: usar WMDD como cota superior inicial en un *branch and bound*, o como punto de partida de una búsqueda local.
6. **Medir bien es parte del análisis.** La brecha porcentual con óptimos cercanos a cero produjo cifras engañosas. Reportar medianas y separar los resultados por tipo de instancia cambió las conclusiones respecto a mirar solo promedios globales.

Referencias

[1] E. L. Lawler. A "pseudopolynomial" algorithm for sequencing jobs to minimize total tardiness. *Annals of Discrete Mathematics*, 1:331–342, 1977.

[2] J. K. Lenstra, A. H. G. Rinnooy Kan y P. Brucker. Complexity of machine scheduling problems. *Annals of Discrete Mathematics*, 1:343–362, 1977.

[3] W. E. Smith. Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, 3:59–66, 1956.

[4] J. R. Jackson. Scheduling a production line to minimize maximum tardiness. Research Report 43, UCLA, 1955.

[5] M. Held y R. M. Karp. A dynamic programming approach to sequencing problems. *Journal of SIAM*, 10(1):196–210, 1962.

[6] M. L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, 5.ª ed., 2016.

[7] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*. MIT Press, 4.ª ed., 2022.